*Module 5 — MCP Servers*

Lesson 5.4

# Resources, tools, and prompts

*Three MCP primitives. Tools do; resources are; prompts template. Pick the right one and your server feels right.*

## Why it matters

Most servers reach for "tool" for everything because tools are the most familiar shape. That conflates reads and writes, hides data behind a function call, and makes it harder for Claude to know what's available. Using all three primitives correctly makes your server discoverable and predictable.

## The three primitives

### Tools — actions with side effects, or computations

Tools are invoked with arguments and return a value. Use for:

- **Actions** — "create a ticket", "send a Slack message", "deploy"
- **Computations** — "convert this timestamp", "search this corpus"
- **Anything dynamic** — the result depends on the arguments

Example:

```
@mcp.tool()
def create_linear_ticket(title: str, description: str, project_id: str) -> dict:
    """Create a ticket. Returns the ticket ID and URL."""
    ...
```

### Resources — read-only addressable data

Resources have a URI and return content when fetched. Use for:

- **Documents** — files, notes, design docs
- **Records** — a row from a DB by ID, a Slack thread by ID
- **Anything addressable** — Claude can reference by URI in conversation

Example:

```
@mcp.resource("ticket://{ticket_id}")
def ticket_content(ticket_id: str) -> str:
    """Get the full markdown body of a ticket."""
    ...
```

The difference from a tool that "fetches" something: resources are listable. Claude can ask "what resources exist?" and get a catalog. A `get_ticket(id)` tool can't be listed in the same way.

### Prompts — reusable templated prompts

Prompts are server-defined named prompts Claude can be asked to load. Use for:

- **Workflow templates** your team has standardized
- **Format-heavy prompts** that benefit from server-side construction
- **Prompts that need server data** — the prompt can be parameterized

Example:

```
@mcp.prompt()
def standup_summary(team: str, date: str) -> str:
    """Generate a standup summary for the team."""
    members = get_team_members(team)
```

```
    activity = get_activity(team, date)
    return f"Write a standup summary for {team} on {date}. Team: {members}. Activity:
    {activity}."
```

The prompt is a function that returns a string; the string becomes the prompt template.

## Which primitive for which goal

| Goal | Primitive |
|---|---|
| "Give Claude a thing it can do" | Tool |
| "Give Claude a body of content it can browse" | Resources |
| "Give Claude a starting prompt for a common workflow" | Prompt |
| "Give Claude a search interface" | Tool (the query is dynamic) |
| "Give Claude a way to read a known document" | Resource (URI is stable) |

## A common error: tool-as-resource

```
# Anti-pattern
@mcp.tool()
def get_employee(employee_id: str) -> dict:
    ...
```

This works, but it's wrong. Claude can call it (good), but it can't list "what employees exist" (bad). A resource:

```
@mcp.resource("employee://{employee_id}")
def employee(employee_id: str) -> dict:
    ...

@mcp.tool()
def list_employee_ids(department: str | None = None) -> list[str]:
    ...
```

Now there's a tool to discover and a resource to fetch. Cleaner mental model, easier for Claude to use.

## Tool descriptions are critical

The docstring of a tool is what Claude reads when deciding whether to call it. Bad descriptions cause:

- Claude not calling a tool when it should
- Claude calling the wrong tool because two have similar descriptions
- Claude calling a tool with wrong arguments

Write descriptions for Claude:

- State precisely what the tool does
- State when to use it

- State what it returns
- State side effects ("this writes to production")

```
@mcp.tool()
def execute_sql(query: str) -> dict:
    """
    Run a read-only SQL query against the production replica.
    Use for ad-hoc data questions. Do NOT use for schema changes or writes.

    Returns:
        {"rows": [...], "row_count": int, "elapsed_ms": int}

    Errors:
        Raises if the query contains DDL or DML. Replica may be ~30s behind primary.
    """
```

## Resource patterns

### Static catalog

A resource scheme where listing gives a finite catalog:

```
@mcp.resource("project://{slug}")
def project_doc(slug: str) -> str:
    ...

# implement list_resources to return all known project slugs
```

### Templated, generated on demand

A resource that's computed when fetched:

```
@mcp.resource("report://daily/{date}")
def daily_report(date: str) -> str:
    return generate_report(date)  # built fresh each fetch
```

### Reference into an external system

A resource that fetches from elsewhere on demand:

```
@mcp.resource("ticket://{id}")
def ticket(id: str) -> dict:
    return linear_client.get_ticket(id)
```

## Prompts: when they earn their keep

Prompts are the least-used primitive but shine when:

- Your team has a standard workflow ("triage an issue") that benefits from a starting prompt with server-side data baked in
- The prompt body changes based on server state and would be tedious to construct client-side
- You want a discoverable list of canonical workflows ("what can this server help me do?")

If your prompts are static and don't need server-side data, a custom slash command is simpler — keep prompts for cases that actually need server logic.

## Try it

In your Python MCP server from 5.3:

1. Refactor `search_notes` and `get_note` into a tool + resource pair. Make sure listing resources gives a sensible catalog.
2. Add a prompt: `daily_journal_prompt(date)` that returns a prompt template parameterized with the date.
3. Use each from a Claude Code session and notice the different ergonomics.

## Gotchas

- **Tools that "get" data should usually be resources.** Notice the pattern, refactor.
- **Don't expose huge resource lists.** If listing returns 10,000 entries, Claude can't usefully browse. Provide a search tool instead.
- **Prompts can't be parameterized at call time in arbitrary ways.** They're templates with named slots, not arbitrary functions.
- **Resource fetches count toward context.** A 50 KB resource read takes 50 KB of context.

## Further reading

- MCP spec on primitives: https://modelcontextprotocol.io/docs/concepts
- 5.5 Auth and security

## Next

→ 5.5 Auth and security